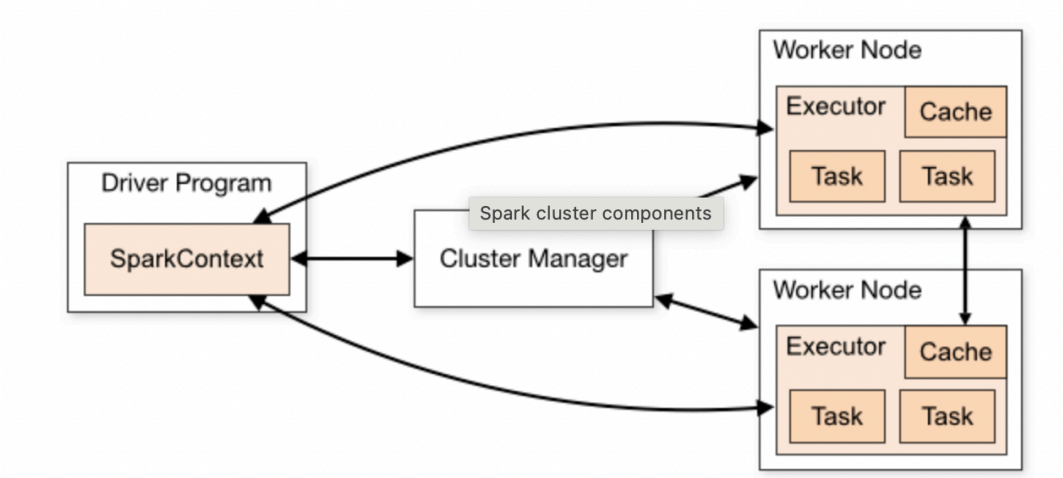
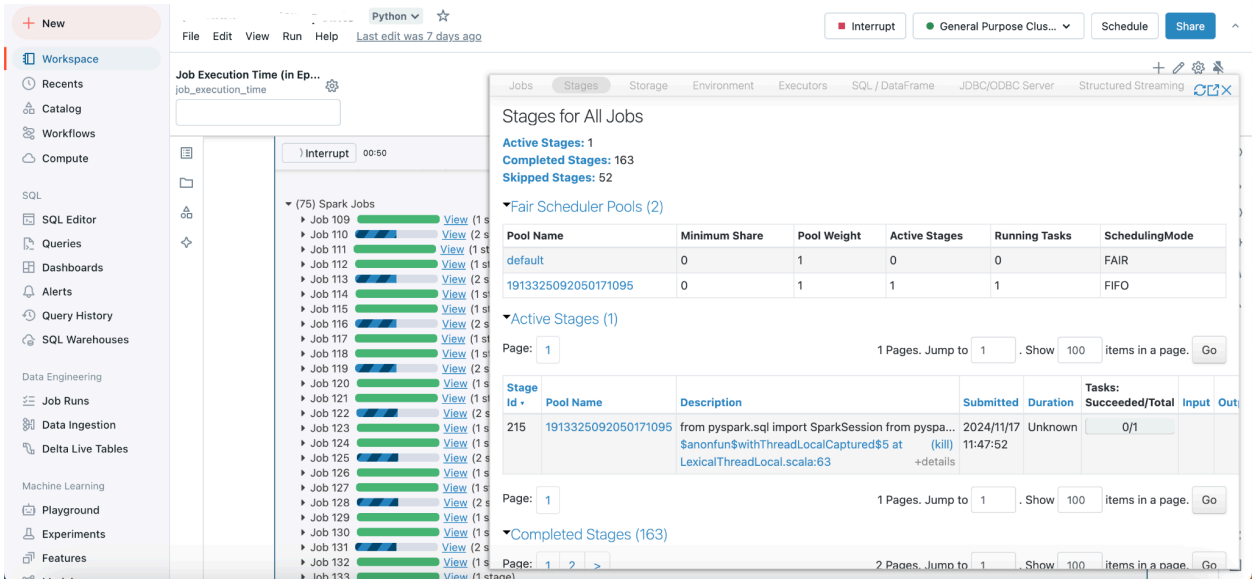
PySpark

Batch Pipelines





- Spark turns the user's data processing commands into a Directed Acyclic Graph, or DAG, which determines the execution plan, i.e what tasks are executed on what nodes and in what sequence.
- Spark runs in a distributed fashion by combining a Driver core process that splits a Spark application into tasks & distributes them among many Executor processes that do the work. Driver sends tasks to executors via Cluster Manager.
- OpenSource vs Databricks managed Spark:

- **Native Spark** (open-source):
  - If running mode: --deploy-mode cluster:
    - spark-submit job → Resource Manager
      - submit-job always talks to RM first.
      - You must configure beforehand: driver-memory, driver-cores, num-executors, executor-memory, executor-cores, dynamic allocation, cluster manager settings, etc., when submitting a job.
      - Local machine can disconnect ("fire and forget") - Prod use.
    - Resource Manager launches Driver (random node inside the cluster)
    - Driver starts running your Spark app
    - Driver requests executors from Resource Manager
    - Resource Manager allocates executors on worker nodes
    - Executors register back with Driver
    - **Note**: If you request more Spark executor cores or instances than your infrastructure actually has, the resource manager would still allocate what is physically available rather than crashing the system.
  - If running mode: --deploy-mode client:
    - Driver runs on your machine
      - submit-job starts driver locally, then driver calls RM (next point).
      - In client mode, you "skip" the step of asking the Resource Manager to find a home for the Driver because it's already living on your machine.
      - Local machine must stay online for the job to finish - Local use.
    - Driver talks to Resource Manager
    - Resource Manager allocates executors

    - **Databricks (managed Spark)**: User submits job → Databricks handles almost EVERYTHING. You only configure high-level things.
- SparkContext vs SparkSession:
    - SparkContext: Only ONE per JVM; Core connection to cluster; Low level.
    - SparkSession: Multiple possible; Wrapper around SparkContext; Abstracted level.
        - getOrCreate() → returns existing session if present (same object)
        - newSession() → creates a new SparkSession (different object, same SparkContext)
        - spark.stop() → required to fully reset everything
- Data Processing:
    - **Resilient Distributed Dataset (RDD)**: Low-level abstraction.
        - Compile-time checking before the job runs.

```
For ex., type-safety checked:
val df = Seq((1,"Alice")).toDF("id", "name")
df.select("nam").show())
// Typo in column name: "nam" instead of "name".
// Compilation error thrown in start, before job runs
```

        - NOT Catalyst/Tungsten engine optimised hence comparatively slow.
        - Use - Low level data transformations, custom partitioning data etc.
    - **DataFrame**: Medium-level abstraction.
        - NO compile-time checking.
        - Catalyst/Tungsten optimised.
        - Use - Analytical queries, SQL-like aggregations, joins, filters.
    - **Dataset**: High-level abstraction.
        - Compile-time checking.
        - Catalyst/Tungsten optimised.
            - Hence best of dataframe & RDD.
        - Use - Type safety + functional + SQL benefits (Scala/Java only).

- Components: Driver node creates jobs/stages/tasks, and workers execute or process data partitions.
    - **Application** - A user program built on Spark using its APIs. It consists of a driver program and executors/workers on the cluster.
    - **Job** - Refers to a sequence of transformations on data in response to a Spark Action. Each job is converted into a DAG with execution plan/stages.
        - Execution: Sequential, unless you explicitly trigger multiple actions asynchronously (e.g., via threads, async APIs, or multiple Spark contexts).
        - Actions: collect(), count(), take(n)/head(n)/tail(), first(), foreach(func), foreachPartition(func), saveAsTextFile(), saveAsParquet(), saveAsTable(), reduce(func), fold(), aggregate(), countByKey(), collectAsMap(), toPandas(), write.format(‘’).save(), show(), rdd.takeSample(), rdd.saveAs...(), spark.sql("SELECT ...").show(), writeStream.start().
        - We see from earlier that writes trigger a job. But in case of: spark.sql("SELECT ...") or spark.read.parquet("path") will only build a logical plan, no action. If you couple it with .show(), then action is triggered.
            - **Exception**: Generally, when reading data, spark would know the schema like if reading parquet data, then from metadata of file, schema is known. But assume, reading CSV (comma-separated-values) then spark may inferSchema. If it does then spark.read.csv("file.csv", inferSchema=True) could trigger 1 job unlike our previously discussed point.
        - Note:
            - `count()` is an action that triggers a job. Computation happens on executors, and only a final aggregated count is returned to the driver → memory safe but requires full data scan.
            - `collect()` brings all data to the driver → can cause OOM, should be avoided for large datasets.

        - `head(n)` is an action that retrieves first `n` rows and may stop early after scanning enough partitions → more efficient than full scan.
        - `tail(n)` is inefficient in distributed systems since Spark may need to scan the entire dataset → generally avoided.
    - **Stage** - Each job gets divided into smaller sets of tasks called stages, in which a sequence of transformations can be executed in a single pass, i.e., without any shuffling of data. The boundary between two stages is due to data shuffling. Transformations never "bring data" immediately (they're lazy - will discuss). But they define how tasks and stages will run once an action triggers them.
        - Execution: Sequential
        - Transformations:
            - **Wide (creates a new stage)**: reduceByKey(), groupByKey(), groupBy(), join(df, on="id", how="left"), distinct(), repartition(), sortBy(), orderBy(), sortWithinPartitions(), cogroup(), cartesian(), aggregateByKey().
            - **Narrow**: map(), filter(), union(), flatMap(), withColumn(), drop() / alias(), coalesce(), mapPartitions(), sample(), mapValues(), keyBy(), select() / where() / limit(), cache() / persist() / checkpoint(), createOrReplaceTempView().
            - Note:
                - If you do: coalesce(numPartitions, shuffle=True) - then it becomes a wide transformation. Acts like repartition only.
                - limit(n) is tricky: narrow if no global order, wide if combined with orderBy.

- **Task** - A task is the smallest unit of execution in Spark, executed on a single core of an executor. Each stage is divided into multiple tasks, where each task processes one partition of the data. A task runs all transformations in that stage on its assigned partition. The number of tasks equals the number of partitions, and the number of tasks that can run in parallel depends on the total available cores in the cluster.
    - Execution: Parallel
- Example:

```
spark = SparkSession.builder \
    .appName("MultipleJobsStagesTasksExample") \
    .master("local[4]") \
    # local[4] → uses 4 cores → at most 4 tasks can run in
parallel
    # local[*] → uses all available cores on the machine
    # NOTE:
    # - This controls PARALLELISM (concurrent task execution), NOT
number of partitions directly
    # - spark.default.parallelism may be derived from cores if not
explicitly set
    .config("spark.executor.memory", "2g") \
    .config("spark.driver.memory", "1g") \
    .config("spark.sql.shuffle.partitions", "16") \
    # Default for spark.sql.shuffle.partitions = 200 → overridden
to 16 (affects shuffle stages like groupBy)
    .getOrCreate()

# DATA SOURCE
df = spark.createDataFrame(data, columns)

# NOTE:
# - createDataFrame from in-memory data → usually creates 1
partition
# - "size / 128MB" rule DOES NOT apply here
# - That rule applies when reading from files (parquet/orc/etc.):
#     partitions ≈ total_data_size /
```

```
spark.sql.files.maxPartitionBytes (default 128MB)

# WIDE TRANSFORMATION → causes shuffle
df = df.repartition(8, "group_id")
# After this → df has 8 partitions

# TRANSFORMATIONS (lazy)
df_filtered = df.filter(col("amount") > 20)
# narrow transformation → no shuffle → pipelined
df_mapped = df_filtered.withColumn("amount_plus_5", col("amount")
+ 5)
# still narrow → stays in same stage

# WIDE TRANSFORMATION → causes shuffle
df_grouped = df_mapped.groupBy("group_id") \
.agg(avg("amount_plus_5").alias("avg_amount"))

# ACTION 1 → triggers Job 1
df_grouped.show()

# ACTION 2 → triggers Job 2 (recomputes lineage since no cache)
df_mapped.count()

# ------------------------------------------------
# Job 1 → df_grouped.show()

# Stage 1:
# createDataFrame → repartition(8, group_id) → shuffle (map
side)
# tasks = initial partitions of df (likely 1, since
createDataFrame)

# Stage 2:
# repartition(8, group_id) → shuffle (reduce side)
# + filter + withColumn → narrow transformations (pipelined)
# tasks = 8 (new partitions after repartition)
```

```
# Stage 3:
# groupBy → shuffle (map side)
# tasks = 8 (input partitions)

# Stage 4:
# groupBy → shuffle (reduce side / aggregation)
# tasks = 16 (controlled by spark.sql.shuffle.partitions)

# Driver collects limited rows (default 20) for show()

# ------------------------------------------------
# Job 2 → df_mapped.count()

# Stage 1:
# createDataFrame → repartition(8, group_id) → shuffle (map
side)
# tasks = initial partitions (likely 1)

# Stage 2:
# repartition(8, group_id) → shuffle (reduce side)
# + filter + withColumn → pipelined
# tasks = 8

# Final:
# Each partition computes partial count
# → tree aggregation → final result returned to driver

# ------------------------------------------------
# NOTES:
# 1. Partition vs Parallelism:
#    - Partitions → define number of tasks
#    - Cores (local[4]) → define how many tasks run in parallel
# 2. File-based reads:
#    - partitions ≈ data_size / 128MB (default)
#    - controlled by: spark.sql.files.maxPartitionBytes
# 3. In-memory (createDataFrame):
#    - usually 1 partition unless explicitly repartitioned
```

```
# 4. repartition(n):
#    - full shuffle
#    - sets number of partitions to n
# 5. spark.sql.shuffle.partitions:
#    - controls number of partitions AFTER shuffle (e.g., groupBy)
# ------------------------------------------------
```

- Spark does lazy evaluation. Transformations (filter, map, groupBy) do NOT execute immediately. They build a logical plan (DAG). Only when you call an action like show(), count(), write(); Spark triggers execution.
    - Plans are of 2 types: Logical + Physical.
    - Evaluation Engines:
        - **Catalyst** (Spark's query optimization engine that converts a logical plan into an optimized physical execution plan using rule-based and cost-based optimizations)
        - **Tungsten** (Spark's physical execution engine that focuses on CPU and memory efficiency using code generation, off-heap memory, and vectorized processing)
    - Plan lifecycle:

```
Unresolved Logical Plan
        ↓   (Catalyst: Analysis)
Analyzed Logical Plan
        ↓   (Catalyst: Logical Optimization)
Optimized Logical Plan
        ↓   (Catalyst: Physical Planning + CBO)
Physical Plan
        ↓   (Tungsten: Whole-Stage Codegen)
Optimized Physical Plan (with fused operators)
        ↓   (Tungsten: Execution Engine)
Execution:
  - Off-heap memory management
  - UnsafeRow (binary format)
  - Cache-aware execution
  - Vectorized processing (batch execution)
        ↓
Tasks run on Executors (1 task per partition)
```

```
        ↓
Result returned to Driver
```

- **Unresolved Logical Plan**: Just the code what you wrote.
- **Analyzed Logical Plan**: Resolves: Column names, Data types, Table references
- **Optimized Logical Plan**: Rule-based and cost-based optimizations applied by Catalyst like:
  - Predicate Pushdown: filter early → less data read
  - Projection Pushdown (Column pruning): read only required columns
  - Constant Folding: simplify expressions at compile-time rather than execution-time. Eg: SELECT * FROM users WHERE age > (10 + 5) --> SELECT * FROM users WHERE age > (15); Other ex: SELECT name FROM users WHERE age > 30 AND 1 = 1 --> Removing the 1=1.
  - Null Propagation: If it detects that a transformation will inevitably result in null because one of the constants is null, it can skip the calculation entirely.
  - Join Optimization: Choosing joins (broadcast, shuffle, etc. - will discuss); Avoiding unnecessary shuffles.
  - Operation Reordering: Eg: map → join → filter to filter → map → join to reduce dataset size
  - Pipeline Fusion: Here Spark fuses multiple consecutive operations (like filter, map, and project) into a single, highly optimized function. So instead of moving data from one function to the next, data stays in CPU registers as long as possible.
- **Physical Plan**: Spark chooses how to execute based on cost (Cost Based Optimizer). Cost includes: Data size (bytes), Row count, Shuffle cost (network I/O), CPU cost, Memory usage. Config to enable it: spark.conf.set("spark.sql.cbo.enabled", True).
- **Optimized Physical Plan**:
  - Whole-Stage Code Generation: Instead of processing data row-by-row with multiple function calls, Spark: generates

one optimized Java program for the entire stage - via Janino (JIT compiler) and runs directly on JVM. So: No virtual function calls, No object creation per row, Tight CPU loop → better performance.

```
Eg:
> Old (Iterator Model -- slow)
Each operator runs separately
Too many function calls
Row → Filter → Row → Map → Row → Output
while (iterator.hasNext()) {
   Row row = iterator.next();
   if (row.value > 10) {
       output.append(row.value * 2);
   }
}
> New (Whole-stage codegen -- fast)
Spark merges all operations into one loop
for (int i = 0; i < numRows; i++) {
   if (column[i] > 10) {
       output[i] = column[i] * 2;
   }
}
```

- Off-Heap Memory Management: Instead of storing data as Java objects (which cause GC overhead): Spark stores data in raw binary format outside JVM heap. Uses low-level APIs like sun.misc.Unsafe.
- Cache-Aware Execution (UnsafeRow): Data is stored in contiguous memory blocks. Instead of: Row → object → pointer → value; Spark uses: Flat memory → direct offset access. So no pointer chasing and faster data access.
    - UnsafeRow is the internal binary format Spark uses to represent a row of data in memory.
- Vectorized Processing: Instead of processing 1 row at a time: Spark processes batches of rows (column-wise). Files like parquet/ORC are perfect for vectorized processing due to columnar format.

    - **Execution Runtime**:
        - Adaptive Query Execution (AQE) happens at runtime, after the physical plan is created. Enable AQE via: spark.conf.set("spark.sql.adaptive.enabled", "true"). It does:
            - Dynamic Join Strategy Change: Say planned was Shuffle Join but may switch to Broadcast Join if data is small.
            - Coalesce Shuffle Partitions: Reduce 200 → 20 partitions to avoid too many small tasks.
            - Skew Join Optimization: Detects skewed partitions (very large ones); Splits them → avoids slow tasks
            - Better Aggregation Decisions.
- Partition is a logical chunk of data that Spark processes in a task on a core. Ideal number: 2-3 partitions on a core. Default partitioning: automatically partitions data based on the underlying file system block size — typically 128 MB or 256 MB per block in HDFS/S3. Eg: Reading 3 files of 1 GB → ≈ 8 × 3 = 24 partitions (1 GB / 128 MB ≈ 8).
    - Types of Partitioning:
        - **Hash Partitioning**: Takes your key, applies a hash function, and assigns it to a partition by: partition = hash(key) % numPartitions. Deterministic distribution but if one key repeats a lot, that partition maybe skewed. Eg: join(), groupByKey(), reduceByKey(), distinct(). Code ex:

```
rdd = sc.parallelize([("A", 1), ("B", 2), ("A", 3), ("C", 4)],
numSlices=3)
grouped = rdd.groupByKey() # triggers hash partitioning
Other ex:
Say data: 1,2,3,4,5,6… If hash partition, i.e hash(id)%4 (Assume
4 constant used) then data would go accordingly to:
1 in partition p1
2 in partition p2
3 in partition p3
4 in partition p0
5 in partition p1, etc…
```

        - **Range Partitioning**: Sorts keys and divides them into contiguous ranges, so each partition gets a range of keys, say:

partition 0: keys < 100, partition 1: 100–199, etc. Eg: sortByKey(), Range-based joins, etc. Code ex:

```
data = [(5, "A"), (1, "B"), (10, "C"), (20, "D"), (100, "E")]
rdd = sc.parallelize(data, 2)
sorted_rdd = rdd.sortByKey(numPartitions=3) # internally uses
RangePartitioner
```

- **Custom Partitioning**: By extending the Partitioner class (RDD-level only, not DataFrame).

- Adjust partitions using:
  - **repartition**(n) – increase/decrease partitions with full shuffle (expensive but creates balanced, equal-sized partitions).
  - **coalesce**(n) – reduce partitions without full shuffle (cheap but may create uneven partitions; good for writing smaller outputs). It has a shuffle=True/False flag.
  - Sidenote: You may see a similar partition function called: **partitionBy()**. It is NOT the same as repartition. partitionBy() is used only during writes. Physical data layout on disk is affected (organizing files), not execution-time partitioning. Useful for query pruning. It triggers a shuffle. Eg:

```
df.write.partitionBy("country").parquet("/path").
This creates directory structure like:
/path/
country=US/
country=IN/
country=UK/
```

- Partition strategy:
  - Count: If high -> excessive overhead in managing many small tasks, If low -> underutilised cores in cluster.
  - Size: If large -> long computation time, slow write times, executor/worker OOM, If small -> slow read time downstream, large task creation overhead, driver OOM
  - Avoid partition on high-cardinality keys (e.g., user_id) → causes small files problem, skew, catalogue explosion, and slow query planning.

- Caching: Cache intermediate DataFrames to avoid recomputation across stages. Although the number of stages/tasks created would be in expected lines and not reduced, but skipped. Cache vs Persist vs Checkpoint:
    - **Cache**: Stores the DataFrame in memory (RAM) by default. It's equivalent to: df.persist(StorageLevel.MEMORY_ONLY). To remove cache ~ df.unpersist()
    - **Persist**: More flexible version of cache() — you can choose where and how to store data: Memory only, Memory + Disk, Disk only, Serialized, etc.
    - **Checkpoint**: It's like a savepoint. Breaks the lineage of a DataFrame (cuts off its dependency graph); Writes data to disk (HDFS/S3) as a new materialized copy; Used for fault tolerance and state management (streaming or long transformations). Forces a job execution. Use a checkpoint if lineage is too long. Eg:
      # Set the checkpoint dir
      spark.sparkContext.setCheckpointDir("/mnt/checkpoints")
      # Doing the checkpoint
      df.checkpoint()

```
Eg:
from pyspark.sql.functions import col
df = spark.read.parquet("big_data.parquet")
df1 = df.filter(col("amount") > 100)        # narrow
df2 = df1.withColumn("x", col("amount") * 2)  # narrow
# df2.cache() # caching if required
# Reused multiple times
df3 = df2.groupBy("category").count()       # shuffle
df4 = df2.groupBy("region").avg("x")        # shuffle
df3.show()   # Action 1 → Job 1
df4.show()   # Action 2 → Job 2
## Without cache df2 is recomputed twice for df3/df4
## With cache, work is reduced inside stages
```

- Joins:
    - **Broadcast Hash Join**: Send the small table to every machine, and join it with chunks of the large table locally. No need to move the large table around. Related config: spark.sql.autoBroadcastJoinThreshold

```
How it works (example)
Data:
Small table (users)        Large table (transactions)
id | name                  id | amount
1  | A                     1  | 100
2  | B                     2  | 200
                           1  | 300
Execution:
Small table is copied to all executors:
Executor 1 → gets full small table
Executor 2 → gets full small table
Each executor processes its chunk of large table:
Executor 1:
  (1,100), (2,200)
Executor 2:
  (1,300)
For each row in large table:
Look up matching id in small table
Join immediately
Result:
(1,A,100)
(2,B,200)
(1,A,300)
```

    - **Shuffle Hash Join**: Move both tables so that same keys come together, then: Build a quick lookup (hash map) from smaller data; Then for each row in larger data → find matches. Less used these days.

```
How it works (example)
Data:
df1 (small)              df2 (large)
```

```
id | val                    id | val
1  | A                      1  | X
2  | B                      2  | Y
1  | C                      3  | Z
Step 1: Shuffle both tables by id
Partition 1 (id=1)          Partition 2 (id=2,3)
df1: (1,A), (1,C)           df1: (2,B)
df2: (1,X)                  df2: (2,Y), (3,Z)
Now matching keys are together
Step 2: Inside each partition build lookup from smaller table;
HashMap:
1 → [(A), (C)]
Now process larger table rows:
Row: (1,X)
→ find id=1 in hashmap
→ matches (A), (C)
Result:
(1,A,X)
(1,C,X)
...
```

- **Sort Merge Join**: Move both tables so the same keys come together, then sort both sides; then walk through them together and match rows. Related config: spark.sql.join.preferSortMergeJoin

```
How it works (example)
Data:
df1                         df2
id | val                    id | val
1  | A                      1  | X
2  | B                      2  | Y
1  | C                      3  | Z
Step 1: Shuffle both tables by id; Now partitions contain same
keys
Step 2: Sort inside each partitions
Partition A:
df1: (1,A), (1,C)
```

```
df2: (1,X)
Partition B:
df1: (2,B)
df2: (2,Y), (3,Z)
# Already sorted but Spark ensures sorting
Step 3: Walk through both (like merging sorted arrays)
Compare:
1 vs 1 → Say in partition A -> 1A-1X, 1C-1X -> match → output.
Other id's no match.
2 vs 2 → Say in partition B -> 2B-2Y -> match → output. Other
id's no match.
Result: (1,A,X), (1,C,X), (2,B,Y)

**Note**:
- A Spark partition is NOT just raw bytes like a file block. A
logical chunk of data processed by one task, exposed as an
iterator of rows.
- Spark sees something like: Partition A: df1_iter =
Iterator[(1,A), (1,C)] and df2_iter = Iterator[(1,X)
- It is not merged into one single contiguous array. It's two
independent streams of data in partition. Contiguous bytes idea
may come from thinking in terms of HDFS/storage, but during
execution, Spark converts that into row iterators / UnsafeRows.
```

- **Bucketing** = pre-shuffling data by a key and storing it that way on disk.
  Data is divided into a fixed number of buckets using:
  bucket = hash(join_key) % num_buckets.
  Rows with the same key → always go to the same bucket.
  - It happens only during write, i.e saved as tables. Syntax: .bucketBy(...).saveAsTable(...). It is not applied on normal dataframe in memory generally.
  - Bucketing is a storage-level optimization, not runtime transformation. Eg:

```
df1.write.bucketBy(4,
"id").sortBy("id").saveAsTable("bucketed_df1")
```

```
df2.write.bucketBy(4,
"id").sortBy("id").saveAsTable("bucketed_df2")
Now: Both tables have 4 buckets; Same hashing logic → id = 1 goes
to the same bucket number in both tables.
```

    - If both tables are bucketed on the same key, and we have the same number of buckets, then Spark can do bucket-to-bucket join.
    - Bucketing may not always avoid shuffling.
- Skewness in data: When some keys/partitions have much more data than others, causing: Few tasks run very slow (stragglers) + Other tasks finish quickly → resources idle. To solve, different ways like Broadcast join, Increase partitions, Adaptive Skew Join, Preaggregation (apply filters early), etc. We'll discuss more on Salting.
    - **Salting**: Break one heavy key into multiple smaller keys. So the heavy key split into multiple partitions. Load gets distributed.

```
Eg:
# Create a "Salted Key". Instead of '1', we get '1_0', '1_4',
'1_9', etc.
df_large_salted = df_large.withColumn(
    "salted_dept_id",
    concat(col("dept_id"), lit("_"), floor(rand() * 10))
)
salt_array = array([lit(i) for i in range(10)])
# Explode the small table so each dept_id exists 10 times
df_small_exploded = df_small.withColumn("salt",
explode(salt_array)).withColumn("salted_dept_id",
concat(col("dept_id"), lit("_"), col("salt")))
—-----------------------------------------------------
Sidenote on: What explode does:
It converts one row with an array/map into multiple rows.
Input:
id | items
1  | [A, B, C]
2  | [D, E]

from pyspark.sql.functions import explode
df.select("id", explode("items").alias("item"))
```

```
Output:
id | item
1  | A
1  | B
1  | C
2  | D
2  | E
-------------------------------------------------------

Coming back, to salting: Assume data:
Large table (orders)
dept_id | order
1       | o1
1       | o2
1       | o3
1       | o4   ← 🔥 heavy key
2       | o5

Small table (dept info)
dept_id | name
1       | Sales
2       | HR

What goes wrong without salting: During join: All dept_id = 1 →
go to SAME partition.
One task gets: (1,o1), (1,o2), (1,o3), (1,o4)
Other tasks are almost empty
One slow task → overall job slow (data skew)

Hence we add salting.
Step 1: Add salt to LARGE table: salt ∈ {0,1}.
dept_id | order | salt
1       | o1    | 0
1       | o2    | 1
1       | o3    | 0
1       | o4    | 1
```

```
2       | o5    | 0
Heavy key (1) is now split: (1,0) and (1,1). Load distributed.
Problem now: Small table is still:
1 | Sales
2 | HR
It does NOT have salt. So join like: (1,0) ≠ (1) ❌, (1,1) ≠ (1) ❌
Join will FAIL / miss data.

Step 2: Expand SMALL table: Duplicate small table for all salts;
It is intentional.
dept_id | name  | salt
1       | Sales | 0
1       | Sales | 1
2       | HR    | 0
2       | HR    | 1

Step 3: Join on (dept_id, salt). Now matching works:
(1,o1,0) → matches (1,Sales,0)
(1,o2,1) → matches (1,Sales,1)
(1,o3,0) → matches (1,Sales,0)
(1,o4,1) → matches (1,Sales,1)

Result:
dept_id | order | name
1       | o1    | Sales
1       | o2    | Sales
1       | o3    | Sales
1       | o4    | Sales
2       | o5    | HR

After join: Remove the salt: drop("salt")
```

- Executors (Fat/Tiny):
    - **Fat Executors**: Few executors with more cores & memory per executor

    - **Tiny Executors**: Many executors with fewer cores & memory each
- **Spark UDFs**: UDF (User-Defined Function) is a custom function written by the user to extend Spark's capabilities beyond the built-in functions provided by Spark. Say you have to cater some complex business logic or nested JSON parsing, etc. It runs row by row on executors hence slow.
    - **Native Spark (No UDF):**
        - Execution: Logical plan → optimized by Catalyst
        - Converted to **compiled JVM code (WholeStageCodegen).** Runs fully in JVM
        - Runs as a tight **loop inside the JVM.** Vectorized (batch processing). CPU-cache friendly. Predicate pushdown possible.
        - **Eg:** df.filter(df.age > 30)
    - **Spark (With UDF)**:
        - Execution: JVM → serialize → Python → execute → serialize → JVM
        - Problems:
            - Serialization Overhead as data moves from JVM → Python → JVM. Uses Pickle. Happens repeatedly per batch
            - Cross language: JVM and Python = separate processes. Communication via sockets/pipes
            - No Catalyst Optimization. Spark cannot see inside UDF. So cannot apply push filters down, reorder operations, etc. Also since it executes on worker nodes, logs are not visible in driver node, but only worker nodes.
            - No Code Generation: Native → compiled Java. UDF → interpreted Python
            - Breaks Vectorization: Native → batch processing. UDF → often row-wise or less efficient batching
    - Pandas UDF could be a better alternative. As it uses **Apache Arrow.** Columnar data transfer. Vectorized execution. Faster than Python UDF, although slower than native spark.

- Spark always deals with serialization at some level, but DataFrames/Datasets minimize and optimize it, while RDDs rely on generic serialization. RDDs store data as JVM/Python objects requiring heavy serialization, while DataFrames use Tungsten's binary format, reducing serialization overhead and enabling optimized execution.
- Data Warehouse vs Lake vs Lakehouse:
    - **Warehouse**: Structured, curated data for analytics & BI.
    - **Lake**: Store everything (raw data) cheaply.
    - **Lakehouse** (Modern approach): Combine Data Lake + Data Warehouse.
    - Others:
        - **Schema-on-Read** vs **Schema-on-Write**
            - Schema-on-write → validate before storing (warehouse)
            - Schema-on-read → apply when reading (lake)
        - **ETL** vs **ELT**
            - ETL → Transform before loading (warehouse)
            - ELT → Load first, transform later (lake/lakehouse)
        - Medallion Architecture (Lakehouse): Bronze → raw data, Silver → cleaned, Gold → aggregated
- Copy-on-Write (COW) and Merge-on-Read (MOR) are two strategies for handling row-level updates (like UPDATE or DELETE) in modern data lake formats like Apache Iceberg, Hudi, and Delta Lake
    - **COW**:
        - Mechanism: Total Replacement. When a row is modified, the system locates the file containing that row and rewrites a brand-new version of the entire file with the changes applied.
        - Write Speed: Slow and expensive. Even a tiny change forces a rewrite of a large file (High Write Amplification).
        - Read Speed: Fast and efficient. The query engine reads clean, finished files with no extra "math" required.
        - Best For: Read-heavy workloads where data doesn't change often (e.g., historical archives).
    - **MOR**:

        - Mechanism: Log-based Changes. Instead of rewriting the big data file, the system keeps the original file as-is and writes a small "sidecar" file (a log of deletes or updates).
        - Write Speed: Very fast. It only records the delta (the change), not the whole file (Low Write Amplification).
        - Read Speed: Slower. The query engine must read the base data PLUS apply the log of changes at runtime to show the final result.
        - Best For: Write-heavy or streaming workloads (e.g., CDC pipelines where data flows in constantly).
- External vs Delta tables:
    - **External**:
        - Data stored outside Spark metastore, only metadata is stored.
        - Points to data in: S3 / HDFS / ADLS. Dropping table → data NOT deleted.
        - No ACID guarantees. Works with formats like Parquet, CSV, JSON.
    - **Delta**:
        - It is a storage format (table format), not a table type, i.e: Parquet files + _delta_log (metadata layer). It does NOT just "virtually apply changes". Delta is not a row-level mutation system rather a file-level versioning system.
        - spark.read.format("delta").load(...) reads _delta_log and figures out valid files for the current version and reads only them.
        - Stores data as parquet files only internally. _delta_log stores metadata like: File additions/removals, schema, transactions, etc. You can perform DELETE/UPDATE row level operations (plain parquet external tables don't allow that). Eg:

```
-> deltaTable.delete("date < '2024-01-01'")
-> deltaTable.update(
     condition="status = 'pending'",
     set={"status": "'processed'"}
   )
```

        - Other properties of delta tables:

        - ACID Transactions: Safe concurrent reads/writes
        - Schema Enforcement & Evolution: Prevents bad writes, Supports schema changes
        - Time Travel: Ability to query a table exactly as it existed at a specific point in history. Eg: spark.read.format("delta").option("versionAsOf", 5).load(path)
        - Change Data Feed (CDF): Read only changed data. Eg: spark.read.format("delta").option("readChangeFeed","true").option("startingVersion",2).option("endingVersion", 3)
        - Optimistic Concurrency Control: Multiple writers allowed, Conflict detection via versions. “Last writer wins” only if there is no conflict.
        - MERGE operation: Upserts data. Eg: MERGE INTO target USING source ON target.id = source.id WHEN MATCHED THEN UPDATE WHEN NOT MATCHED THEN INSERT
        - INSERT OVERWRITE operation: Replaces entire table. Eg: INSERT OVERWRITE TABLE user_data SELECT * FROM new_snapshot;
    - Problems: **Small Files Problem** is a very common issue in Delta Lake. When you are doing: micro-batch/stream writes, frequent merge/DML operations, over-partitioning, etc. This causes: Metadata bloat (Delta tracks every file in its transaction log, which becomes huge overtime), Increased IO overhead, Higher costs.
    - Solution:
        - **OPTIMIZE table**: Does compaction (bin-packing small files → large files)
        - **Z-ORDER**: It is a multi-dimensional clustering technique for data files - for delta lake, apache iceberg, apache hudi. It co-locates related column values together in the same set of files to make filtering and range scans faster. Normal external tables lack a metadata layer (like Delta's Transaction Log or

Iceberg's Manifests) to store the min/max statistics created by the Z-Order process. Without these statistics, the query engine cannot perform "data skipping," making the physical Z-Order arrangement of rows useless. Eg: OPTIMIZE events ZORDER BY (country, date)

```
Eg:
SELECT * FROM transactions WHERE country = 'IN' AND event_time
BETWEEN '2025-10-01' AND '2025-10-10';

Delta physically rearranges data on disk so that rows with similar
country and event_time values end up close together (in the same
files).
So when you filter by these columns, Delta needs to read fewer
files — because the relevant data is tightly clustered instead of
being scattered across many files.

Query: OPTIMIZE transactions ZORDER BY (country, event_time);

Internally, it interleaves the bits and stores, like:
country bits:   0 1 1 0 0 1
event_time bits:1 1 1 0 1 0 0 1 0 1 0 1
Z-order bits:   0 1 1 1 0 0 1 1 0 0 1 0 1 0 1…
Rows are sorted by this interleaved single composite Z-order bit
key.
Z-Ordering > Sorting by One Column
```

- **VACUUM**: Deletes old files (physically). Affects time travel though. Eg: VACUUM table RETAIN 168 HOURS

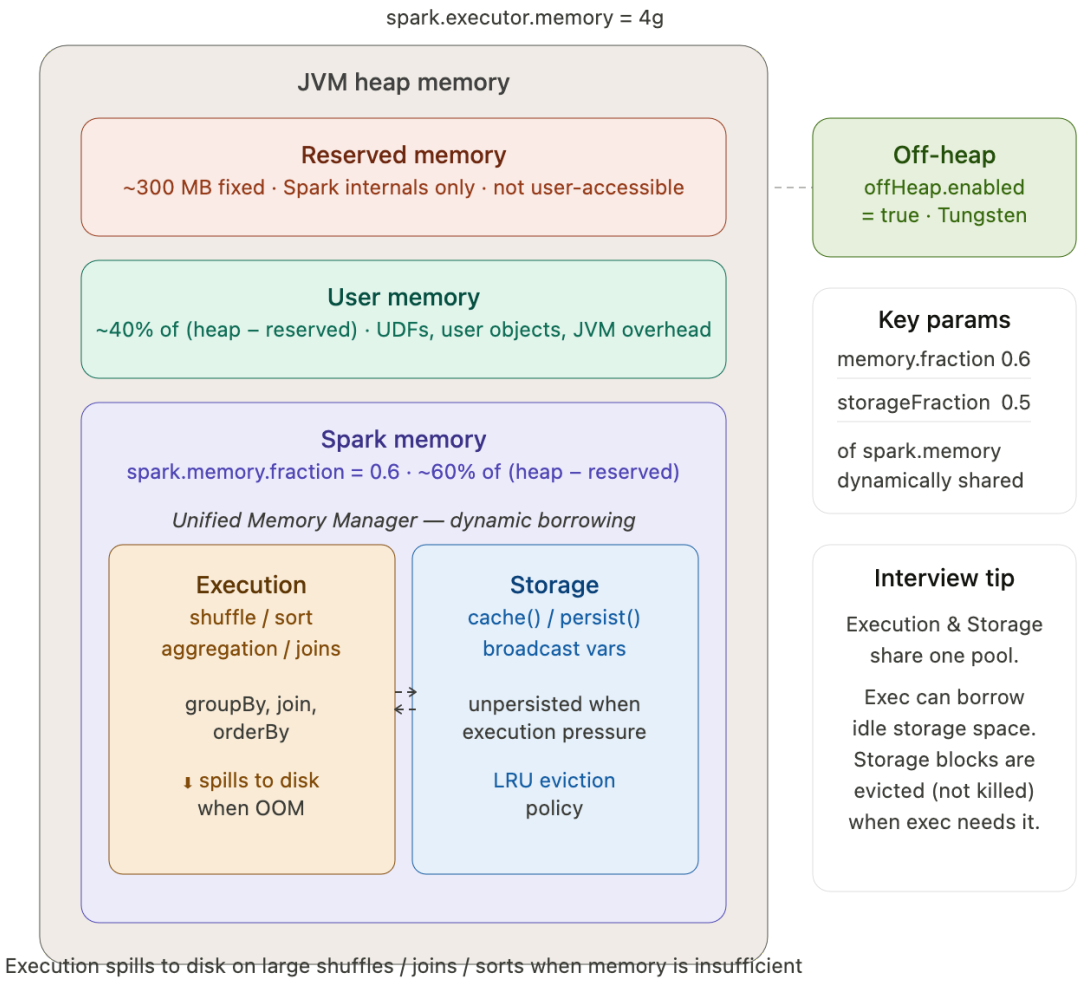
■ Solved inefficiencies:

- **Deletion Vectors (DVs)**: Traditionally, Delta used **Copy-on-Write**—if you deleted one row, the entire Parquet file containing it had to be rewritten. Deletion Vectors shift this to a Merge-on-Read paradigm. Eg: delta.enableDeletionVectors

- **Liquid Clustering** is the successor to Partitioning and Z-Order. It is a flexible, "self-tuning" layout that automatically groups related data. How it works: It uses a tree-based algorithm (often Hilbert Curves) to organize data incrementally. Unlike Z-Order, it only clusters the newly arrived data rather than rewriting the entire table every time.
- For columns with high cardinality (like email, user_id, or ip_address) that don't cluster well, Delta uses Bloom Filter Indexes.
    - **Bloom filter** is a space-efficient, probabilistic data structure used to determine if an element is a member of a set. It is exceptionally fast and memory-efficient but comes with a unique trade-off: it can tell you if an item is "definitely not" in a set or "possibly" in a set.
      How it works: "Instead of storing the data itself, we store its footprint in a bit array of size.
        - Insertion: We run an input through independent hash functions. Each hash outputs an index, and we set those specific bits to 1.
        - Lookup: We hash the query the same way. If we find even a single 0 at any resulting index, we know the item was never added. If all are 1, it's a 'maybe' due to potential bit-overlap from other items (as other items could have similar bits turned on) —which we call a false positive." Say inserting apples, searching for oranges, etc.
    - It performs well at scale:
        - Space: You can track millions of items in just a few megabytes because you aren't storing strings or objects, just bits.

    - Speed: Complexity is O(k). Whether you have a thousand items or a billion, the lookup time is the same because it's just 'k' simple math operations.
- **Spark Memory split** in workers: Assume a single 4GB worker:

  Executor Heap Memory
  ├── Reserved Memory (~300MB)
  ├── User Memory
  └── Spark Memory
      ├── Execution Memory
      └── Storage Memory



    - Memory hierarchy — heap splits into 3 parts: Reserved (~300 MB hard floor), User (~40% of remaining), and Spark Memory (~60% of remaining via `spark.memory.fraction`).
    - Unified Memory Manager: Execution and Storage are not fixed splits — they share a pool. Execution can borrow idle storage space. When execution pressure is high, cached blocks get LRU-evicted (not killed), freeing space. Storage, however, cannot forcibly reclaim memory held by active execution tasks.
    - Spill to disk — happens automatically when execution memory is exhausted during shuffle/sort/join. Performance drops but no OOM.
    - Off-heap — optional, outside JVM GC, enabled via `spark.memory.offHeap.enabled=true`. Used by Tungsten's binary processing to reduce GC pauses.
- Other tips:
    - UNION removes duplicate rows, UNION ALL does not. There is a performance hit when using UNION instead of UNION ALL, choose based on use case.
    - foreach runs function per row (use for debugging, not for I/O as could lead to millions of calls) and foreachPartition runs function per partition on executor (ideal for external system writes).
    - EXISTS/NOT EXISTS (Stops evaluating as soon as a match is found, faster) > IN/NOT IN (Subquery may be fully evaluated, slower on large datasets).
    - UDF and foreachPartition process data row by row, but they operate at completely different layers of Spark's architecture — and have very different purposes. UDF transform data inside the Spark SQL/DataFrame engine. Operates on data values (columns, rows) as part of a transformation (select, withColumn, filter, etc.) on executors. The output of a UDF is a new column value, part of the logical plan. Whereas foreach() perform actions with side effects, often for external I/O. Also runs on executors, but outside Spark's SQL planner. Operates on raw data (RDD/Row objects) per partition. Used after all transformations are done.

- Spark is faster than Hadoop/MapReduce due to in-memory data processing (in Hadoop every stage writes to disk) so disk I/O reduced, can spill memory to disk if required to handle large datasets, uses Catalyst optimizer, etc.
    - Eg:
        - Hadoop: step → disk → step → disk
        - Spark: step → memory → step → memory
    - Spark + HDFS >> MapReduce + HDFS.
    - Hadoop is better if very big datasets bigger than memory need to be processed.
- **Photon** is Databricks' next-generation vectorized execution engine, written in C++, designed to replace many of Spark's traditional JVM operators. Photon uses: SIMD vectorization, tighter memory layout, CPU instruction-level optimizations (AVX-512, etc.). Photon accelerates: File Scans (Parquet, Delta), Projection, Filter, Aggregations, Joins (hash, sort-merge). Where Photon does NOT work: Python / Scala UDFs, Complex nested types in some cases, MLlib operators, Graph workloads.
- Use KryoSerializer instead of Java serializer as when Spark runs distributed jobs: Data is sent between executors (e.g., for shuffles, caching, broadcasts). Each time, Spark must serialize (convert objects into byte form) to transfer them across the network or store them in memory/disk. Hence performance matters. Eg: .config("spark.serializer", "org.apache.spark.serializer.KryoSerializer")
- Parquet vs Avro vs ORC:

Feature	Parquet (Columnar)	Avro (Row-based)	ORC (Columnar)
Best For	Spark, general analytics, AWS Athena	Kafka, high-speed ingestion	Apache Hive, ACID transactions

Query Speed	**Fastest** for reading specific columns	Slow for analytics (must read whole row)	Fast, especially for filtering with indexes
Compression	High (column-specific)	Moderate	**Highest** (stripe-level optimization)
Schema Evolution	Basic (mostly adding columns)	**Superior** (renames, deletes, defaults)	Moderate

- Note:

```
--Note--
Parquet format supports Bloom filters, but they are not enabled by default. Standard Parquet metadata stores the minimum and maximum values for each row group. If you query WHERE id = 'abc', and the row group's range is aaa to zzz, the engine must scan that group because abc falls inside the range.
To use Bloom filters, you must enable them specifically during the write process so Spark can build the index and store it in the Parquet file's metadata.
Eg: df.write.mode("overwrite") \
  .option("parquet.bloom.filter.enabled", "true") \
  .option("parquet.bloom.filter.columns", "user_id, txn_id") \
  .option("parquet.bloom.filter.expected.ndv#user_id", "100000") \
  .parquet("path/to/output")
```

-

Structured Streaming [High-level basics]

- **Core mental model**: Spark treats streaming as incremental batch processing on an unbounded DataFrame. The engine reads new data on each trigger, builds an incremental logical plan, executes a Spark DAG, optionally updates state, writes to a sink, then commits offsets to

checkpoint. The abstraction you work with is a DataFrame that logically grows forever.

- **Pipeline:** Source → Trigger → Transformations → Output Mode → Sink, with Checkpoint running alongside.
- **Driver role:** coordinates triggers, tracks offsets, plans the query, manages checkpoint commits.
- **Executor role:** runs micro-batch tasks, maintains state store partitions, executes transformations.
- **Micro-batch vs Continuous Processing**

Mode	How it works	Latency	Production status
Micro-batch	Batches executed as Spark jobs	100ms–seconds	Standard, full guarantees
Continuous	Long-running tasks, row-by-row	~1ms	Experimental, limited operators

- Even Structured Streaming is fundamentally micro-batch in all real production systems. Continuous mode is rarely used.
- **What happens in each micro-batch trigger:**
  - Read new offsets from source
  - Build incremental logical plan
  - Execute Spark DAG on executors
  - Update state store (if stateful)
  - Write to sink
  - Commit offsets + checkpoint
- Triggers:
  - **Trigger.ProcessingTime("10 seconds")** — fires on fixed interval, default mode.

    - **Trigger.AvailableNow** — (Spark 3.3+) processes all backlog across multiple safe micro-batches then stops; recommended for incremental ETL.
    - **Trigger.Once** — single batch then stops; can OOM on large backlog, now superseded. Continuous — experimental, record-by-record, sub-millisecond latency, avoid in prod.

	Trigger.Once	**Trigger.AvailableNow**
Batch count	Single huge batch	Multiple micro-batches
Large backlog	Can OOM	Safe
Status	Older	Recommended

- Time semantics:
    - **Processing time** — when Spark processed the record.
    - **Event time** — when the event actually occurred at the source. These can differ significantly.
    - Example: user clicked at 10:00, Kafka delayed delivery until 10:15 — event time is still 10:00. Watermarks operate on event time, not processing time. This distinction is critical for correctness of windowed aggregations.
- **Watermarking**
    - withWatermark("event_time", "10 minutes"): Spark tracks the maximum event time seen so far and allows data to arrive up to 10 minutes late relative to that event time. Data older than the watermark is considered too late and may be discarded.
    - Why watermarks matter:
        - Correctness for Append mode on aggregations: Without a watermark, Spark cannot know whether a future late event

might still update an old aggregation/window result. Therefore the result is never considered final, making Append mode invalid for aggregations whose results may still change. With a watermark, Spark knows when a window is sufficiently old and can safely emit the final result in Append mode. (Stateless append queries do not require watermarks because rows are emitted once and never updated.)
        - Bounding state size: Stateful operations maintain intermediate state in the state store. Without watermarks, old aggregation/join state may accumulate forever, eventually causing excessive memory usage, GC pressure, spills, or OOM errors.
        - State cleanup / eviction: As the watermark advances, Spark evicts expired window state and old stream-stream join state from the state store.
- Window types;
    - **Tumbling window** — fixed size, non-overlapping: Eg: window(col("ts"), "10 minutes")
    - **Sliding window** — overlapping, a record can belong to multiple windows: Eg: window(col("ts"), "10 minutes", "5 minutes")
    - **Session window** — dynamic, closes after an inactivity gap: Eg: session_window(col("ts"), "30 minutes")
- Operations
    - **Stateless** — filter, map, select. No state store touched. Rows are emitted once and never revised. Effectively Append behavior regardless of mode declared; Update mode is technically allowed but behaves identically.
    - **Stateful** — aggregations, windowed ops, stream–stream joins. Uses the StateStore API internally: versioned, checkpointed, one state partition per shuffle partition. Backed by HDFS/S3 (default) or RocksDB (Spark 3.2+, spark.sql.streaming.stateStore.providerClass). RocksDB is strongly preferred for large or long-lived state — avoids JVM GC pressure.
    - **State store internals:** large cardinality aggregation keys → massive state growth → GC pressure → OOM. Optimization levers:

watermarks, careful key design, RocksDB backend, proper partitioning.

- Stream joins
    - **Stream–Static:** join a stream against a static DataFrame. No watermark needed. The static side behaves like a normal batch DataFrame and is joined against each incoming micro-batch.
    - **Stream–Stream:** both sides unbounded. Watermark is mandatory on both sides to bound the join buffer. Without it, Spark must buffer all rows from both sides indefinitely.
- **Message delivery guarantees**: (how a system handles data as it moves from a source to a destination)
    - **At-Most-Once**: Data may be lost but is never duplicated (fastest, lowest reliability).
    - **At-Least-Once**: Data is never lost but may be duplicated (standard for most pipelines).
    - **Exactly-Once**: Data is neither lost nor duplicated (highest accuracy, requires checkpoints/idempotency).
- Output modes
    - **Append** — new rows only. Valid for: stateless ops, and stateful ops with watermark (once window is closed). Illegal on non-watermarked aggregations — Spark throws at runtime.
    - **Update** — only rows changed this batch. Most memory-efficient for aggregations. Not supported by file sinks.
    - **Complete** — entire result table every batch. Requires an aggregation. Expensive for large result sets.
- Source-side rate limiting
    - The old spark.streaming.backpressure.enabled=true is a DStreams config — it does not apply to Structured Streaming. For Structured Streaming, control ingestion rate through source-specific options: Eg: .option("maxOffsetsPerTrigger", 100000) # Kafka; .option("maxFilesPerTrigger", 10) # file source
- Sinks
    - **foreach** — row-level, runs on executors. Not idempotent by default. If a batch fails midway and retries, your writer runs again on already-processed rows. Spark's checkpoint only tracks source

offsets, not what your external writer did. Avoid for production DB writes.
    - **foreachBatch** — batch-level callback. Callback executes on the driver; batch_df operations inside run distributed on executors. batchId is provided — use it to make writes idempotent (e.g. deduplicate on batchId before insert). Exactly-once semantics are achievable if and only if your sink write is implemented idempotently using batchId or transactional logic. Spark cannot magically guarantee external DB semantics.
    - **Built-in sinks:** Kafka (exactly-once natively), Parquet/JSON (micro-batch file writes), memory (testing only).
    - **Delta Lake sink** — ACID transactional writes, exactly-once behavior, supports streaming reads and writes, CDC patterns, and the standard Bronze/Silver/Gold lakehouse architecture. Increasingly the default production sink. Eg: df.writeStream.format("delta").option("checkpointLocation", "...").start("path/to/table")
- **Fault tolerance and exactly-once**: Spark guarantees: exactly-once processing of input offsets (via checkpointed offsets + replayability). End-to-end exactly-once depends entirely on the sink:

Sink	Guarantee
Kafka	Exactly-once (native)
Delta Lake	Exactly-once (transactional)
foreachBatch + dedup	Exactly-once (if implemented correctly)

JDBC / foreach	At-least-once unless you add dedup

- **Checkpointing**: Stores three things: source offsets (e.g. Kafka partition offsets), state store snapshots (for stateful ops), job metadata and progress. Stored on HDFS or S3. Without it, Spark cannot resume from failure.
- **Unsupported / restricted operations**: Global sort on an unbounded stream, certain non-time-bounded stream–stream joins, collect() and similar actions, non-deterministic operations. Root cause: streaming datasets are unbounded so these operations have no valid completion point.
- **Common production problems**: Small file problem from frequent micro-batches writing to object storage. State explosion from high-cardinality keys without watermarks. Checkpoint corruption (requires manual recovery or offset reset). Data skew in stateful aggregations. Kafka partition imbalance causing uneven executor load. Late data silently dropped when it exceeds the watermark — no error, just gone. Long GC pauses with the default JVM state store on large state (fix: RocksDB).
- **Kafka & Spark streaming**: Kafka partitions map to Spark tasks. Parallelism heavily depends on Kafka partition count. Spark stores consumed offsets in checkpoint, not Kafka consumer groups (usually). maxOffsetsPerTrigger controls ingestion rate. Too few Kafka partitions limits streaming parallelism.
-

—————————————————————————————

Hadoop-MapReduce

- Hadoop = HDFS (storage) + MapReduce (compute)
- HDFS (Hadoop Distributed File System — Distributed Storage Layer)
    - Purpose: Reliable, fault-tolerant storage for very large files across a distributed cluster.

    - Architecture:
        - NameNode (Master): Stores metadata — file names, block locations, permissions. Keeps a transaction log (EditLog) and a checkpoint (FsImage). Doesn't store actual data, only where blocks live. Single point of failure in early Hadoop versions (fixed later via Secondary NameNode / Standby NameNode / High Availability mode).
        - DataNodes (Slaves): Store actual file blocks (default size: 128 MB or 256 MB). Send periodic heartbeats and block reports to the NameNode. Perform replication, deletion, and block creation as instructed by the NameNode.
    - Key Concepts:
        - Replication factor: Default = 3 → improves fault tolerance and availability.
        - Rack Awareness: NameNode knows rack topology → places replicas on different racks to balance network efficiency and fault resilience.
        - Read/Write Path: Client → NameNode (for metadata) → Read from nearest Datanodes or write blocks of data.
    - Benefits: High throughput, scalable storage, fault-tolerant, optimized for large sequential reads/writes.
- MapReduce (Distributed Processing Framework)
    - Purpose: Parallel computation model that processes massive data using Map() and Reduce() functions.
    - Flow:
        - Job Submission: Client submits job to JobTracker (Hadoop v1) or YARN ResourceManager (v2).
        - Split Phase: Input is divided into chunks (InputSplits), typically aligned with HDFS blocks.
        - Map Phase: Mapper processes each split → outputs intermediate key-value pairs.
        - Shuffle & Sort: Framework groups all values by key and transfers them to the appropriate reducer.
        - Reduce Phase: Reducer aggregates or combines values per key → produces final output.

    - Key Points:
        - Idempotent: Multiple runs on same data yield same output.
        - Fault Tolerance: If a node fails, tasks are re-run on another node using data replicas.
        - Data Locality Optimization: Tries to schedule Map tasks on nodes where data already resides.
        - Output written back to HDFS.
    - Limitation: High latency for iterative or real-time processing — hence, frameworks like Spark were built to overcome this.

```
Sample Problem:
Count frequency of each word in a large dataset. Step-by-step flow in Hadoop
1. Input Splitting
File stored in HDFS
Split into blocks (e.g., 128 MB each)
Each block → processed by a Mapper
2. Map Phase
Each mapper processes its chunk:
Input: "hello world hello"
Output (key-value pairs):
(hello, 1)
(world, 1)
(hello, 1)
3. Shuffle & Sort (MOST IMPORTANT)
Hadoop groups same keys together across all mappers
(hello, [1,1])
(world, [1])
Happens across network (expensive step)
4. Reduce Phase
Reducer aggregates:
(hello, 2)
(world, 1)
Final Output: Stored back in HDFS
```

- YARN (Yet Another Resource Negotiator — Cluster Resource Management Layer)

- Purpose: Decouples resource management from job scheduling, making Hadoop cluster multi-tenant and more efficient.

Miscellaneous(s)

Question1: Assume daily data volume to be 1TB/day from Kafka. Discuss about infra requirements if you were to ingest/parse data and store in S3 - in daily basis (batch) or near real-time basis (streaming).

Answer:

Common Assumptions

Parameter	Value	Notes
Daily data volume	1 TB/day (≈ 1000 GB)	Uncompressed JSON input
Average record size	1 KB	≈ 1 billion records/day
Storage layer	HDFS / S3 / Delta Lake	
Compression after parse	Parquet (≈ 5× smaller)	
Cluster type	Spark on YARN or Kubernetes	
Task	JSON parsing → schema mapping → write Parquet/Delta	

Batch Job (Run Once per Day):

- Workload Pattern
    - One Spark job runs once a day.
    - Reads 1 TB JSON → parses → writes to Parquet.
    - Target: finish in ~1 hour.
- JSON Parsing Throughput
    - Average: 30 – 50 MB/sec per vCPU.
    - Assume 40 MB/sec/vCPU for estimation.
    - Formula: Total time (sec) = (1000 GB × 1024 MB/GB) / (Throughput × vCPUs)
    - Target time = 3600 sec → vCPUs ≈ (1000 × 1024) / (40 × 3600) ≈ 7 cores (ideal)
    - Add ~5× overhead for Spark shuffles, GC, I/O, etc. → ≈ 40–50 vCPUs needed realistically.
- Suggested Infra (Batch): Note that if you're fine with 2–3 hr runtime, halve the cores (~20–25 vCPUs).

Resource	Estimate
Executors	10 – 12
Cores / executor	4 – 5
Total vCPUs	40 – 50
Memory / executor	16 – 24 GB
Total memory	200 – 250 GB
Cluster nodes	~5–6 × m5.4xlarge (AWS)

Cost	~ $3–4 per hour of runtime

Near Real-time Streaming Job:

- Workload Pattern
    - Continuous Kafka ingestion rate: 1 TB / 24 h = 1000 GB / 86400 s ≈ 11.6 MB/sec
    - Goal: process each micro-batch within seconds or a minute.
- Efficiency
    - Structured Streaming is ~2–3× less efficient than batch due to micro-batching and checkpointing.
    - Rule of thumb: ~2 vCPUs per MB/sec input rate
    - At 11.6 MB/sec input: 11.6 × 2 vCPUs ≈ 23 vCPUs
    - Add 50 % headroom → ≈ 35–40 vCPUs total (continuous).
- Suggested Infra (Streaming): Note that streaming runs 24×7, unlike batch.

Resource	Estimate
Executors	8 – 10
Cores / executor	4
Total vCPUs	32 – 40
Memory / executor	16 – 24 GB
Total memory	160 – 200 GB
Cluster nodes	~4–5 × m5.4xlarge (AWS)

Cost	$3–4/hr × 24 h = $75–100/day

Comparison Summary

Aspect	Batch (Daily)	Streaming (Near Real-time)
Data processed	1 TB once/day	Continuous 1 TB/day
Compute needed	~40–50 vCPUs (1 hr run)	~35–40 vCPUs continuous
Memory	~200–250 GB	~160–200 GB
Cost	~$3–4/hr (only 1 hr/day)	~$75–100/day
Latency	Up to 1 day	Seconds – minutes
Complexity	Easier	Higher (checkpointing, watermarking)

Key Takeaways

- Batch = cheaper, simpler if 1-day delay acceptable. Streaming = real-time insights but higher cost (~20–30× more).
- JSON parsing is CPU-intensive — prefer Avro or Parquet ingestion.
- Autoscaling or serverless Spark (Databricks, EMR on EKS) reduces idle cost.
- Can I use less infra for streaming to save cost?
    - Yes — if your streaming job does very little per-record work, one or two vCPUs may keep up with a modest Kafka throughput. But if your streaming job performs CPU-heavy JSON parsing, joins,

windowing, stateful operations, or frequent checkpoints, you need many more cores to avoid lag and operational issues.

- Trade-offs when you reduce infra
  - Increased processing latency — microbatches take longer to finish → higher end-to-end latency (seconds → minutes → hours).
  - Backpressure and Kafka lag — consumer lag will grow if processing rate < input rate; lag must be stored in Kafka (requires sufficient retention).
  - Larger state / checkpoint growth — slower processing increases state growth and checkpoint sizes, slowing recoveries.
  - Spill to disk / GC pressure — insufficient memory leads to disk spill and longer task times; long GC pauses may cause executor failures.
  - Lower fault tolerance and recovery speed — on failure, recovery needs more time if checkpoints are large and cluster is small.
  - Higher operational complexity — must monitor lag, tune partitions, tune microbatch durations, autoscale carefully.
  - Potential for data loss if retention insufficient — if Kafka retention expires before backlog is processed.
- Example scenarios (for 1 TB/day → 11.6 MB/s)
  - Scenario A — Light work (minimal parsing):
    - per_core_MBps = 12 MB/s
    - required_cores = 11.6 / 12 ≈ 1.0
    - headroom 2× → allocated_cores ≈ 2 vCPUs. When appropriate: messages are already compact (AVRO/Protobuf), transforms are trivial, no stateful operations.
  - Scenario B — Medium work (JSON parsing + enrichments)
    - per_core_MBps = 4 MB/s
    - required_cores = 11.6 / 4 ≈ 2.9

        - headroom 2× → allocated_cores ≈ 6 vCPUs
          When appropriate: parsing JSON, some UDFs, light joins, small state windows.
    - Scenario C — Heavy work (stateful joins, large windows, heavy UDFs)
        - per_core_MBps = 1 MB/s
        - required_cores = 11.6 / 1 ≈ 11.6
        - headroom 2× → allocated_cores ≈ 24 vCPUs
          When appropriate: large keyed state, big aggregations, frequent checkpoints, complex UDFs.
    - Note: memory per executor should be aligned to avoid spills (e.g., 16–32 GB per executor depending on JVM overhead and state size). Also distribute workload across many Kafka partitions to parallelize.

Question2: Why RAM faster than Disk (HDD/SSD)?

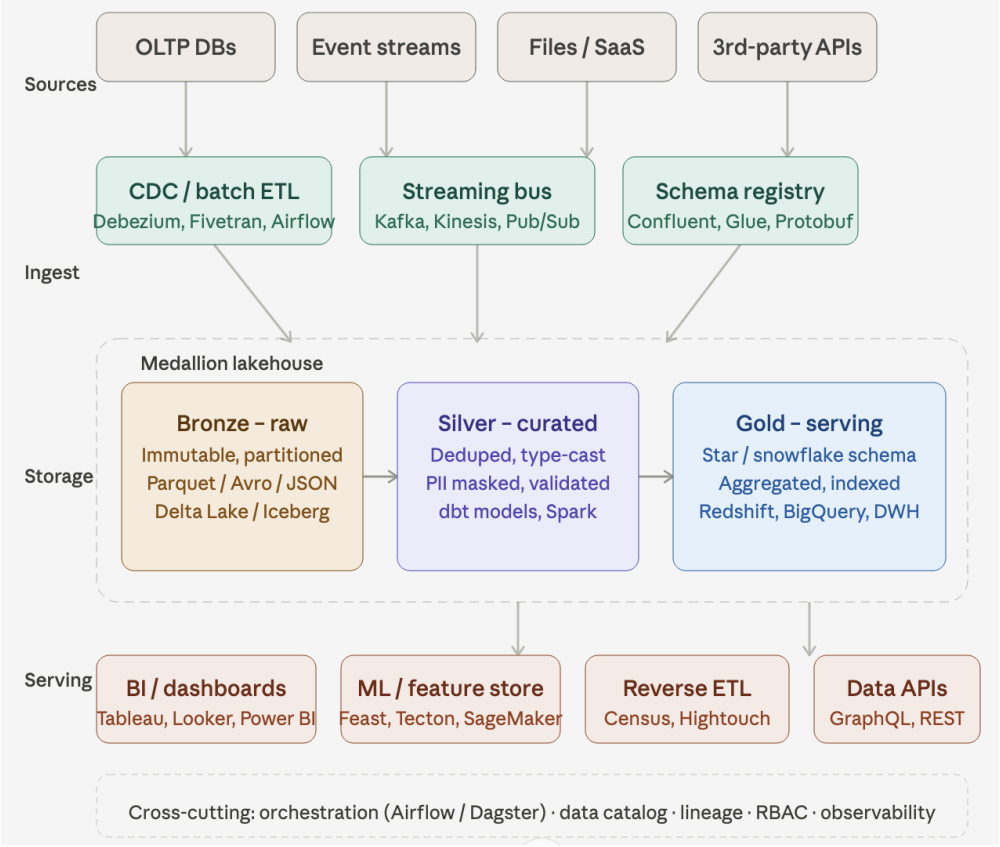
Answer:

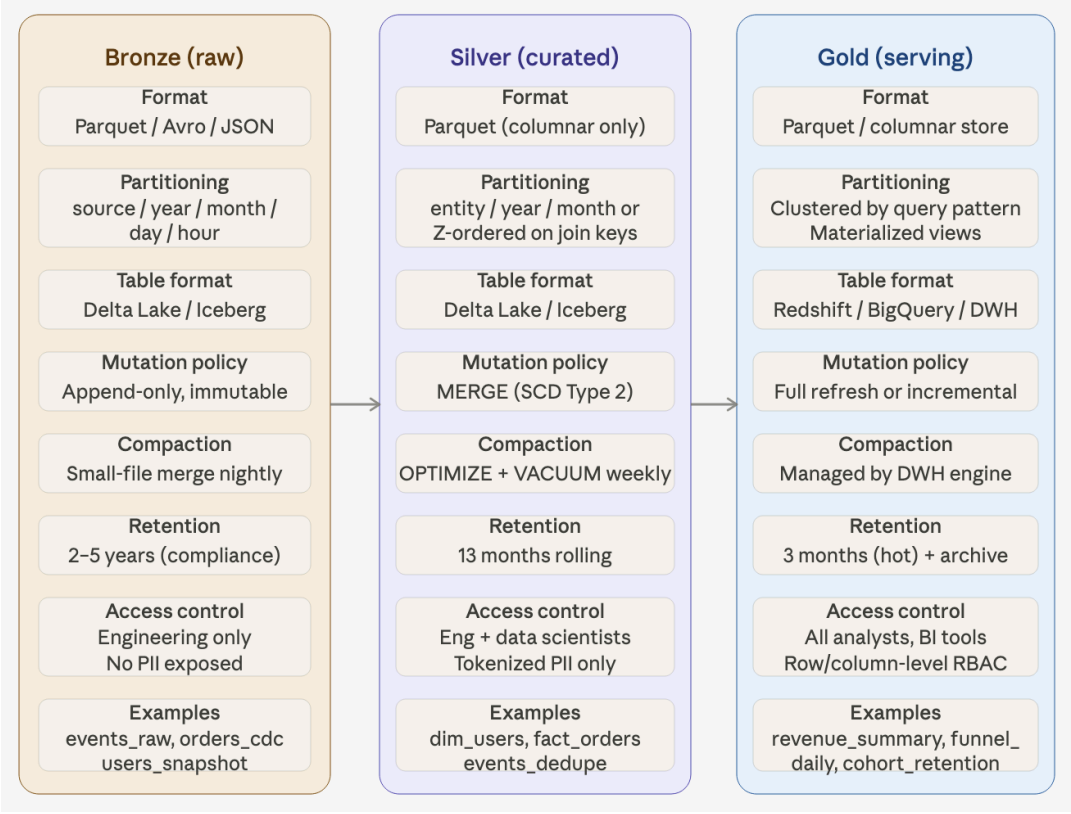
- **RAM (Memory)**: Fully electronic (no moving parts). Data stored using electrical charge in memory cells (transistors + capacitors)
    - Each bit: Charged → 1, Not charged → 0
    - Access: Directly connected to CPU via memory bus. CPU can read/write almost instantly
    - Result: Access time: nanoseconds (10^{-9} sec), Very high bandwidth (GB/s)
    - Nature: Volatile → data lost when power off, Built for speed
- **HDD (Hard Disk Drive - Disk)**: Structure: Spinning platters (glass/aluminum base), Coated with magnetic material, Read/write head moves over surface. Data stored: Surface divided into tiny magnetic regions (bits).
    - Each region: Magnetized one way → 1, Opposite direction → 0.
    - Access: To read data: Disk must spin, Head must move (seek) to correct track, Wait for correct sector to rotate under head

    - Result: This causes: Seek time + rotational latency, Time in milliseconds (10^{-3} sec)
    - Nature: Persistent (non-volatile), Optimized for storage capacity & cost, not speed
- **SSD (Solid State Drive - Disk)**
    - No spinning, no mechanical parts
    - Data stored as electrical charge in flash memory cells
    - Faster than HDD, but still slower than RAM

Question3: Design data warehouse system.

Answer:





Question: What improvements have been made in the Spark 4.x version?

Answer:

Spark 4.x mainly improved: Structured Streaming state management, low-latency real-time streaming, Spark Connect, SQL capabilities, PySpark performance, and observability/debugging. Major highlights include: transformWithState API, AQE for streaming, Real-Time Mode, VARIANT type support, and production-ready Spark Connect.
